

relatório-compilador

Ludmila Maria

October 2025

1 Introduction

Objetivo: entra um arquivo .asm com instruções em assembly, o compilador passa tudo para binário e devolve um arquivo .bin.

Explicação bloco-a-bloco:

Listing 1: Exemplo de código C

```
1 #define WORD_SIZE 8
2 #define MIN_BINARY_LINES 128
```

O `#define` é uma diretiva de pré-processamento da linguagem C. Isso significa que, antes da compilação do código, todas as ocorrências dessas palavras são substituídas pelo valor definido.

A constante `WORD_SIZE` define o tamanho da palavra (1 byte), enquanto `MIN_BINARY_LINES` define o tamanho do arquivo binário de saída. Essa definição garante que o arquivo `.bin` seja sempre uma imagem de memória com tamanho fixo, o que é importante para o uso em VHDL.

Listing 2: Exemplo de código C

```
1 typedef struct {
2     const char* name;
3     const char* opcode;
4     int num_arguments;
5 } Instruction;
```

O `struct` é utilizado para criar a estrutura de dados `Instruction`.

O campo `name` armazena o mnemônico da instrução ("ADD", "JMP", "SUB"), o campo `opcode` armazena o código de máquina binário correspondente ao mnemônico, e o campo `num_arguments` indica a quantidade de argumentos que essa instrução espera receber.

Listing 3: Exemplo de código C

```

1
2 const Instruction mnemonics_table[] = {
3     {"INC", "00000001", 1}, // Increment
4     {"DEC", "00000010", 1}, // Decrement
5     {"NOT", "00000011", 1}, // Bitwise NOT
6     {"JMP", "00000100", 1}, // Unconditional Jump
7     {"ADD", "00010000", 2}, // Addition
8     {"SUB", "00100000", 2}, // Subtraction
9     {"MUL", "00110000", 2}, // Multiplication
10    {"DIV", "01000000", 2}, // Division
11    {"MOD", "01010000", 2}, // Modulo
12    {"AND", "01100000", 2}, // Bitwise AND
13    {"OR", "01110000", 2}, // Bitwise OR
14    {"XOR", "10000000", 2}, // Bitwise XOR
15    {"NAND", "10010000", 2}, // Bitwise NAND
16    {"NOR", "10100000", 2}, // Bitwise NOR
17    {"XNOR", "10110000", 2}, // Bitwise XNOR
18    {"COMP", "11000000", 2} // Compare
19 };

```

Essa tabela define o conjunto de instruções *assembly* válidas. Ela realiza o mapeamento entre cada instrução e seu respectivo **opcode**, além de especificar a quantidade de argumentos que cada uma deve possuir (1 ou 2).

Listing 4: Exemplo de código C

```

1 const int mnemonics_count = sizeof(mnemonics_table) / sizeof
    (mnemonics_table[0]);

```

Esse trecho serve para realizar uma contagem dinâmica da quantidade de instruções presentes na tabela `mnemonics_table`. Ele divide o tamanho total, em bytes, de todo o array (`sizeof(mnemonics_table)`) pelo tamanho, em bytes, de apenas um elemento da tabela (`sizeof(mnemonics_table[0])`).

Atualmente, esse valor é igual a 16, pois há 16 instruções definidas na tabela.

Listing 5: Exemplo de código C

```

1 const Instruction* getInstruction(const char* name) {
2     for (int i = 0; i < mnemonics_count; ++i) {
3         if (strcmp(mnemonics_table[i].name, name) == 0) {
4             return &mnemonics_table[i];
5         }
6     }
7     return NULL;
8 }

```

Essa é a função de busca. Ela recebe o parâmetro `name` e verifica se ele existe na tabela `mnemonics_table`. Se a instrução for encontrada, a função retorna a linha correspondente da tabela; caso contrário, não retorna nenhum valor.

A função `strcmp` é utilizada para comparar duas strings em C, retornando 0 quando elas são iguais.

Listing 6: Exemplo de código C

```
1 void removeComments(char* line) {
2     char* comment_start = strchr(line, ';');
3     if (comment_start != NULL) {
4         *comment_start = '\\0';
5     }
6
7     line[strcspn(line, "\\r\\n")] = 0;
8 }
```

Essa é a função responsável por remover os comentários do arquivo `.asm` e as quebras de linha (`"\\n"`). A função recebe uma linha do arquivo e utiliza a função `strchr` para localizar a primeira ocorrência do caractere `;` na linha, definindo assim o início do comentário (`comment_start`).

Se o caractere `;` for encontrado, ele é substituído por `'\\0'` para que as funções em C passem a ignorar o restante da linha. Em seguida, a função `strcspn(line, "\\r\\n")` é utilizada para encontrar o índice de `'\\r'` ou `'\\n'`, substituindo-o por um caractere nulo a fim de remover a quebra de linha.

Listing 7: Exemplo de código C

```
1 void removeSpaces(char* line) {
2     int len = strlen(line);
3     while (len > 0 && isspace((unsigned char)line[len - 1]))
4     {
5         line[--len] = '\\0';
6     }
7
8     char* start = line;
9     while (*start != '\\0' && isspace((unsigned char)*start))
10    {
11        start++;
12    }
13
14    if (start != line) {
15        memmove(line, start, strlen(start) + 1);
16    }
17 }
```

Assim como a função anterior, essa função também é usada para limpar a linha, mas, desta vez, ela remove os espaços em branco.

O primeiro **while** elimina os espaços finais (do fim para o início) utilizando a função **isspace**, que identifica se um caractere é um espaço em branco. Se for, o caractere é substituído por `'\0'`.

O segundo **while** identifica os espaços em branco no início da linha, utilizando a variável **start** que avança enquanto encontrar espaços. Se houver espaços no início, a função **memmove** desloca a string para a esquerda.

Observação: o `+1` é usado para garantir que o caractere nulo `'\0'` seja sempre incluído ao final da linha.

Listing 8: Exemplo de código C

```
1 void freeBinaryLines(char** lines, int count) {
2     if (lines != NULL) {
3         for (int i = 0; i < count; ++i) {
4             free(lines[i]);
5         }
6         free(lines);
7     }
8 }
```

Essa função é responsável por liberar toda a memória dinâmica utilizada pelo programa para armazenar as linhas binárias, alocada com **malloc** e **realloc**.

Para evitar que o programa apresente falhas (*crash*), a primeira verificação confere se a linha contém algum conteúdo.

No loop, a função **free** é chamada para liberar cada string individualmente e, em seguida, a própria linha. Isso ocorre porque a memória foi alocada em dois níveis (******), sendo necessário liberar ambos.

Listing 9: Exemplo de código C

```
1 char* padArgument(const char* arg) {
2     int argLen = strlen(arg);
3     int padLen = WORD_SIZE - argLen;
4
5     char* padded = (char*)malloc(WORD_SIZE + 1);
6     if (padded == NULL) {
7         perror("erro ao alocar memória para padArgument");
8         return NULL;
9     }
10 }
```

```

11     if (padLen > 0) {
12
13         memset(padded, '0', padLen);
14
15         strcpy(padded + padLen, arg);
16     } else {
17
18         strcpy(padded, arg);
19     }
20
21     return padded;
22 }

```

Essa função padroniza o tamanho de todos os argumentos binários, completando com zeros à esquerda aqueles que forem mais curtos.

A função inicia calculando o tamanho da string recebida (**argLen**) e quanto falta para completar 8 bits (**padLen**). Em seguida, é alocado um novo bloco de memória para criar a string preenchida (**padded**). Observação: o +1 é utilizado novamente para garantir o caractere nulo '\0' ao final da string.

A função **memset** preenche os zeros à esquerda, e **strcpy** copia o argumento original no final da string, pulando os **padLen** caracteres. O **else** trata o caso em que o argumento já possui 8 bits, ou seja, **padLen** é igual a 0.

Listing 10: Exemplo de código C

```

1  int isBinary(const char* arg) {
2      if (arg[0] == '\0') return 0;
3      for (int i = 0; arg[i] != '\0'; ++i) {
4          if (arg[i] != '0' && arg[i] != '1') {
5              return 0;
6          }
7      }
8      return 1;
9  }

```

Essa função valida um argumento, verificando se ele contém apenas os caracteres '0' e '1'.

A primeira verificação confere se a string está vazia. Em seguida, um loop percorre cada caractere do argumento, garantindo que todos sejam '0' ou '1'.

Listing 11: Exemplo de código C

```

1

```

```

2 int main(int argc, char *argv[]){
3
4     //verifica o de argumentos
5     if (argc != 3) {
6         return 1; // erro
7     }
8
9     printf("Entrada: %s\n", argv[1]);
10    printf("Sa da: %s\n", argv[2]);
11
12    //leitura no arquivo de entrada
13    FILE *inputFile = fopen(argv[1], "r");
14    if (inputFile == NULL) {
15        perror("erro/arquivo de entrada");
16        return 1;
17    }
18
19    //escrita no arquivo de sa da
20    FILE *outputFile = fopen(argv[2], "w");
21    if (outputFile == NULL) {
22        perror("erro/arquivo de sa da");
23        fclose(inputFile);
24        return 1;
25    }

```

Agora entramos na função `main`.

A primeira verificação confere se a função recebeu os dois argumentos necessários na execução do programa: o arquivo `.asm` de entrada e o arquivo `.bin` de saída.

Em seguida, o arquivo `.asm` é aberto para leitura, enquanto o arquivo `.bin` é aberto para escrita.

Listing 12: Exemplo de código C

```

1
2 char buffer[256];
3 int lineCounter = 0;
4 char **binLines = NULL;
5 int binLinesCount = 0;

```

O `buffer` é um espaço de trabalho temporário utilizado para armazenar cada linha lida do arquivo. A variável `lineCounter` é responsável por contar as linhas do arquivo `.asm`.

O ponteiro duplo `**binLines` é utilizado para armazenar as linhas de opcode e argumentos já traduzidas para binário, enquanto `binLinesCount` é responsável por contar a quantidade dessas linhas geradas.

Listing 13: Exemplo de código C

```

1  while (fgets(buffer, sizeof(buffer), inputFile) != NULL)
2      {
3
4          lineCounter++;
5
6          removeComments(buffer);
7          removeSpaces(buffer);
8
9          if (buffer[0] == '\0') {
10             continue;
11         }

```

O loop `while` é executado enquanto o arquivo puder ser lido pela função `fgets`. A cada iteração, o contador de linhas (`lineCounter`) é incrementado.

Em seguida, as funções `removeComments` e `removeSpaces` são chamadas para limpar a linha. Por fim, é feita uma verificação para saber se a linha está vazia; caso esteja, o programa simplesmente passa para a próxima iteração.

Listing 14: Exemplo de código C

```

1  const Instruction* instruction = getInstruction(buffer);
2
3      if (instruction == NULL) {
4          fprintf(stderr, "Erro [Linha %d]: Comando
5              desconhecido '%s'.\n", lineCounter, buffer);
6          fclose(inputFile);
7          fclose(outputFile);
8          freeBinaryLines(binLines, binLinesCount);
9          return 3;

```

Após a limpeza da linha, a instrução é buscada e armazenada chamando a função `getInstruction`. Se a instrução não for encontrada, significa que o comando é desconhecido.

Ao final da execução, os arquivos são fechados e a memória alocada dinamicamente é liberada.

Listing 15: Exemplo de código C

```

1      binLinesCount++;
2      binLines = (char**)realloc(binLines, binLinesCount
        * sizeof(char*));

```

```

3
4     if (binLines == NULL) {
5         perror("Erro ao realocar binaryLines (opcode)")
6         ;
7         fclose(inputFile); fclose(outputFile);
8         freeBinaryLines(binLines, binLinesCount-1);
9         return 10;
10    }

```

O contador `binLinesCount` é incrementado, e o ponteiro `binLines` é redimensionado (expandido) utilizando `realloc`, preservando o conteúdo previamente armazenado.

Se a realocação de memória falhar (`binLines == NULL`), uma mensagem de erro é exibida, os arquivos são fechados e a memória previamente alocada é liberada.

Listing 16: Exemplo de código C

```

1 binLines[binLinesCount - 1] = strdup(instruction->opcode);
2     if (binLines[binLinesCount - 1] == NULL) {
3         perror("Erro no strdup (opcode)");
4         fclose(inputFile); fclose(outputFile);
5         freeBinaryLines(binLines, binLinesCount-1);
6         return 11;
7     }

```

O `opcode` é armazenado. A função `strdup` aloca um novo bloco de memória para o `opcode` e copia seu conteúdo para lá. Esse novo bloco é então armazenado no vetor `binLines`.

Em seguida, é feita uma verificação para garantir que a alocação com `strdup` foi bem-sucedida.

Listing 17: Exemplo de código C

```

1 for (int i = 0; i < instruction->num_arguments; ++i) {
2     char argBuffer[256];
3     int argFound = 0;
4
5
6     while (fgets(argBuffer, sizeof(argBuffer),
7         inputFile) != NULL) {
8         lineCounter++;
9         removeComments(argBuffer);
10        removeSpaces(argBuffer);
11        if (argBuffer[0] != '\0') {
12            argFound = 1;
13            break;
14        }
15    }
16 }

```



```

13     }
14 }

```

Esse é o bloco responsável pela leitura dos argumentos.

O primeiro loop (**for**) garante que seja lido exatamente o número de argumentos que a instrução requer (**num_arguments**).

O loop interno (**while**) é utilizado para encontrar a próxima linha válida do arquivo, já que podem existir comentários ou linhas vazias entre o **opcode** e os argumentos. Dentro desse **while**, a linha é lida e limpa; quando uma linha não vazia é encontrada, o loop é encerrado.

Listing 18: Exemplo de código C

```

1
2 if (!argFound) {
3     fprintf(stderr, "Erro: Fim do arquivo alcançado enquanto
4         esperava um argumento para o comando '%s'.\n",
5         buffer);
6     fclose(inputFile); fclose(outputFile); freeBinaryLines(
7         binLines, binLinesCount);
8     return 4;
9 }
10
11 if (!isBinary(argBuffer) || strlen(argBuffer) > WORD_SIZE) {
12     fprintf(stderr, "Erro [Linha %d]: Argumento invalido '%s'
13         para o comando '%s'.\n", lineCounter, argBuffer,
14         buffer);
15     fprintf(stderr, "Esperado um binario de %d bits.\n",
16         WORD_SIZE);
17     fclose(inputFile); fclose(outputFile); freeBinaryLines(
18         binLines, binLinesCount);
19     return 5;
20 }

```

O primeiro **if** verifica se o **while** anterior falhou em encontrar o argumento. Ou seja, se a variável **argFound** ainda for igual a 0, isso significa que o programa chegou ao fim do arquivo enquanto ainda esperava um argumento para o **opcode**.

O segundo **if** verifica se o argumento é válido, chamando a função **isBinary** e conferindo também se o comprimento do argumento está correto.

Listing 19: Exemplo de código C

```

1
2 char* paddedArg = padArgument(argBuffer);

```

```

3         if (paddedArg == NULL) {
4
5             fclose(inputFile); fclose(outputFile);
6             freeBinaryLines(binLines, binLinesCount
7                 );
8             return 12;
9         }

```

Esse bloco padroniza o argumento chamando a função `padArgument` e, em seguida, verifica se a alocação de memória ocorreu corretamente.

Listing 20: Exemplo de código C

```

1 binLinesCount++;
2 binLines = (char**)realloc(binLines, binLinesCount * sizeof(
3     char*));
4 if (binLines == NULL) {
5     perror("Erro ao realocar binaryLines (argumento)");
6     free(paddedArg);
7     fclose(inputFile); fclose(outputFile);
8     freeBinaryLines(binLines, binLinesCount-1);
9     return 13;
10 }
11 binLines[binLinesCount - 1] = paddedArg;

```

Esse bloco armazena o argumento utilizando a mesma lógica do armazenamento do `opcode` apresentada no bloco 16. No entanto, não é necessário usar a função `strdump`, pois a função `padArgument` já criou uma cópia do argumento. Dessa forma, basta armazenar essa cópia diretamente no `binLines`.

Listing 21: Exemplo de código C

```

1 for (int i = 0; i < binLinesCount; ++i) {
2     fprintf(outputFile, "%s\n", binLines[i]);
3 }
4
5 char zeroPad[WORD_SIZE + 1];
6 memset(zeroPad, '0', WORD_SIZE);
7 zeroPad[WORD_SIZE] = '\0';

```

Esse é o bloco de escrita final. O loop `for` percorre o vetor `binLines` e escreve cada linha no arquivo `.bin`. Fora do loop, é criada uma string de preenchimento composta apenas por 0's, utilizada para garantir que o arquivo atinja o tamanho mínimo definido por `MIN_BINARY_LINES`.

Listing 22: Exemplo de código C

```
1  for (int i = binLinesCount; i < MIN_BINARY_LINES; ++i) {
2      fprintf(outputFile, "%s\n", zeroPad);
3  }
4
5  printf("Compilacao legal. %d linhas de código:.\n",
6         binLinesCount);
7
8  fclose(inputFile);
9  fclose(outputFile);
10 freeBinaryLines(binLines, binLinesCount);
11
12 return 0;
```

Esse último loop completa o preenchimento do arquivo `.bin` utilizando a string de preenchimento criada no bloco anterior. Após isso, a compilação do arquivo chega ao fim.